Proyecto de Programación Fase 1

BlackJack Game

Aline Martinez Mayorga, 120000580

Edson Steve Valle Gómez, 504720842

Kendry Julissa Mejias Chaves, 504700703

Yoycel Ortiz Montenegro, 504650885

Universidad Nacional de Costa Rica

EIF 201: Programación 1

Francisco Coulon

30 de octubre de 2026

Tabla de contenido

Introducción **3**
Definición del problema **4**
¿Qué se debe resolver? 4
Entradas 4
Salidas 4
Alcance 4
Restricciones 5
Reglas del juego **5**
Reglas y Configuración de la Mesa 5
Objetivo del jugador 5
Estructura del tapete y distribución del espacio 6
Sistema de valoración de cartas y conteo de puntos 6
Jerarquía y valor de la baraja 6
Clasificación: manos suaves (soft hands) vs. manos duras (hard hands) 7
Desarrollo cronológico de una ronda de juego 7
Fase 1: Periodo de apuestas iniciales 7
Fase 2: Reparto inicial de cartas 7
Fase 3: Verificación antecedente de blackjack 8
Fase 4: Turno de decisiones del jugador 8
Fase 5: Turno obligatorio de la casa (Crupier) 8
Fase 6: Comparación de puntuaciones y distribución de pagos 8
Análisis de las acciones del jugador 8
Plantarse (Stand) 8
Pedir carta (Hit) 9
Doblar la apuesta (Double Down) 9
Dividir la mano (Split) 9
Apuesta de seguro (Insurance) 9
Tabla de Pagos 9
Ejemplos de manos completas **10**
Ejemplo 1.1: Victoria estándar por puntuación superior 10
Ejemplo 1.2: Victoria por exceso del crupier (Bust del crupier) 10
Ejemplo 2.1: Derrota inmediata por pasarse (Bust del jugador) 11
Ejemplo 2.2: Derrota por puntuación inferior frente al crupier 11
Ejemplo 3: Empate entre el jugador y el crupier 12
Ejemplo 4: Victoria mediante Blackjack natural 12
Ejemplo 5: Dividir o doblar la apuesta 13
Ejemplo 5.1: Dividir cartas (Split) 13
Ejemplo 5.2: Doblar la apuesta (Double Down) 13
Diagramas de clases UML **15**
Diagrama de clases 15
Relaciones de clases 16
Descripción de responsabilidades de las clases 17

Plan de acción..... 18
Minutas de reuniones del Equipo..... 21
Reunión del 4 de octubre de 2026..... 21
Reunión del 20 de octubre de 2026..... 22
Reunión del 22 de octubre de 2026..... 22
Reunión del 27 de octubre de 2026..... 23
Avance de la implementación:..... 23
Aspectos pendientes:..... 24
Evidencias: Programación y pruebas..... 24
Evidencias de ejecución: reparto inicial y presentación de cartas:..... 32

Introducción

El juego Blackjack, también conocido como “veintiuno”, es uno de los juegos de cartas más conocidos en los casinos. Su dinámica es sencilla: el jugador compite contra el dealer y gana si logra una mano de mayor valor sin pasar de 21. Una lógica bien organizada combina las reglas de apuestas, seguro, doblar y dividir, el cálculo de probabilidades y la toma de decisiones por parte del jugador, lo que lo convierte en un problema adecuado para practicar la programación orientada a objetos.

Este proyecto consiste en diseñar, documentar e implementar en C++ una versión del Blackjack que se juega por consola. El juego usa un zapato de 6 mazos de 52 cartas cada uno, teniendo 312 en total, estas se barajan una única vez al inicio y cada jugador comienza con 1000 fichas para apostar contra el dealer. El desarrollo del trabajo se realiza en forma grupal, utilizando Git y la documentación técnica.

Este documento corresponde a la Fase 1 y está organizado en secciones: la definición del problema, las reglas del juego con ejemplos de manos completas, el diagrama de clases UML con las responsabilidades de cada clase, el plan de acción del proyecto, las minutas de las reuniones, el avance de la implementación, con capturas de pantalla de las clases y funcionalidades.

Definición del problema

¿Qué se debe resolver?

Se requiere un programa de consola en C++ que permita simular rondas completas de Blackjack entre uno o más jugadores y un dealer controlado por el programa. El programa debe aplicar fielmente las reglas establecidas: el reparto de cartas, el valor de las cartas, las acciones del jugador, el seguro, el turno del dealer y los pagos, además debe administrar el saldo de cada jugador a lo largo de la partida.

Entradas

- Cantidad de jugadores (mínimo uno y máximo 7) y el nombre de cada uno.
- Apuesta de cada jugador en cada ronda, entre la apuesta mínima y la máxima y sin exceder su saldo.
- Decisión del jugador en cada turno: pedir carta, plantarse, doblar o dividir apuesta (cuando esté disponible hacerlo).
- La decisión sobre el seguro cuando la carta visible del dealer es un As.

Salidas

- La representación en consola de las cartas y las manos de los jugadores y del dealer, con la carta oculta del dealer boca abajo hasta que corresponda revelar.
- Los totales de cada mano y el saldo de fichas de cada jugador.
- Las acciones disponibles en cada turno, según el saldo y las cartas de la mano.
- El resultado de cada mano y el pago que le corresponde a cada jugador.
- Mensajes claros ante una entrada inválida, sin que el programa falle y mensaje de fin de partida cuando el zapato se agota o el jugador se queda sin fichas.

Alcance

- Incluye el zapato de 6 mazos barajados una sola vez, las apuestas, las acciones de carta, plantarse, doblar y dividir, el seguro, el blackjack con pago 3:2 y la lógica fija del dealer de pedir hasta tener 17 puntos o más.
- Los pagos con decimales se redondean hacia abajo a la ficha entera.
- La partida termina cuando el zapato se agota, al finalizar la ronda en curso.
- No incluye interfaz gráfica, juego en red ni persistencia de datos entre partidas.

Restricciones

- Saldo inicial 1000 fichas, apuesta mínima de 10 fichas y máxima de 500 fichas, definidos como constantes fáciles de modificar.
- Programación orientada a objetos, con las clases separadas en archivos .h y .cpp.
- Sin variables globales mutables y sin lógica del juego dentro del main.
- Validación de todas las entradas del usuario.
- Desarrollo colaborativo con Git y compilación con Visual Studio.

Reglas del juego

Reglas y Configuración de la Mesa

El Blackjack es uno de los juegos de cartas con mayor tradición en los casinos internacionales, destacándose por su equilibrio perfecto entre el azar y el reparto de naipes y la aplicación de estrategia matemática basada en probabilidades. El juego se rige por las normas estándar de casino internacional, utilizando de forma predeterminada un calzado (o "zapato") compuesto por seis barajas francesas completas de 52 cartas cada una, lo que representa un total de 312 cartas en juego constante.

En la mesa de juego, cada participante ocupa un puesto de apuesta delimitado virtual o físicamente. El crupier representa a la casa (el casino) y administra la distribución de los naipes desde el zapato, así como la recolección de las apuestas perdedoras y la distribución de las ganancias. Todas las cartas se reparten de manera transparente sobre el tapete de la mesa.

Objetivo del jugador

Es un error común pensar que la prioridad en el juego es sumar exactamente 21. Si bien esa es la mejor combinación posible, el propósito fundamental del jugador es vencer al crupier.

Para alcanzar la victoria en cualquier ronda de Blackjack, la mano del jugador debe cumplir una de las dos condiciones fundamentales siguientes:

- **Puntuación Superior Directa:** Obtener una puntuación total superior a la puntuación acumulada por el crupier al finalizar su turno obligatorio, sin sobrepasar en ningún momento el límite máximo permitido de 21 puntos.
- **Derrota del Crupier por Exceso (Bust):** Mantener una mano válida con una suma igual o inferior a 21 puntos mientras el crupier se excede del límite de 21

puntos en su turno obligatorio de pedir cartas (evento denominado “Bust” o pasarse).

Estructura del tapete y distribución del espacio

El tapete de juego está diseñado funcionalmente para guiar el flujo operacional de las apuestas. Cada posición de jugador incluye:

- **Círculo de apuesta principal:** Un círculo o casilla principal donde se depositan las fichas correspondientes a la apuesta inicial de la ronda.
- **Área de reparto de cartas:** Una zona delimitada frente al jugador donde el crupier deposita las cartas principales boca arriba.
- **Áreas para acciones secundarias:** Casillas adyacentes destinadas a la colocación de fichas adicionales cuando el jugador decide doblar su apuesta o dividir un par en dos manos independientes.
- **Línea de seguro:** Una línea marcada en la parte superior del círculo principal donde se depositan las fichas en caso de optar por la apuesta de seguro.

Sistema de valoración de cartas y conteo de puntos

Jerarquía y valor de la baraja

En el Blackjack, a diferencia de otros juegos de cartas como el póquer, los palos de la baraja (Corazones, Diamantes, Tréboles y Picas) carecen por completo de valor jerárquico o cuantitativo. La valoración de la mano depende exclusivamente de la denominación numérica o facial de cada naipe.

Las 312 cartas utilizadas en un zapato de seis barajas se clasifican estrictamente en tres categorías de valoración:

- **Cartas numéricas (2 al 10):** Conservan exactamente su valor facial nominal. Un 2 equivale a 2 puntos, un 5 vale 5 puntos y así sucesivamente.
- **Cartas con cigura (J, Q, K):** La Jota (J), la Reina (Q) y el Rey (K) tienen un valor fijo e inamovible de 10 puntos cada una. Se puede notar que en cada baraja individual de 52 cartas existen 16 cartas que valen 10 puntos (los cuatro dieces y las doce figuras). En un mazo completo de 6 barajas, hay 96 cartas con valor de 10 puntos.
- **El As (A) comodín dinámico:** El As posee un valor dual de 1 punto o de 11 puntos, según convenga al jugador. Si al contar el As como 11 la mano sobrepasa los 21 puntos, el As cambia de forma inmediata y automática su valor a 1 punto.

Clasificación: manos suaves (soft hands) vs. manos duras (hard hands)

La flexibilidad operativa del As introduce una distinción conceptual decisiva para la toma de decisiones en mesa:

- **Mano suave (soft hand):** Es cualquier mano que contiene un As cuyo valor se está contabilizando provisionalmente como 11 puntos sin que la suma total supere los 21 puntos. Por ejemplo, una combinación de As y 6 se clasifica como un '17 suave' (11 + 6). La característica técnica más importante de una mano suave es su absoluta inmunidad al riesgo de pasarse en el siguiente tiro: si el jugador decide pedir una carta adicional y recibe una figura o un diez (por ejemplo, una Q), el As ajusta dinámicamente su valor a 1 punto, transformando la mano en un 17 duro (1 + 6 + 10 = 17) sin causar la pérdida por exceso.
- **Mano dura (hard hand):** Es toda aquella combinación que no contiene ningún As, o que teniéndolo, éste se ve obligado a contar estrictamente como 1 punto para evitar que el total exceda de 21. Ejemplos típicos incluyen un 10 + 7 (17 duro), un 9 + 8 (17 duro) o un As + 6 + 9 (16 duro: 1 + 6 + 9). Las manos duras comprendidas entre los 12 y los 16 puntos representan las situaciones de mayor riesgo en el juego, ya que pedir una carta adicional comporta una alta probabilidad estadística de sobrepasar los 21 puntos.

Desarrollo cronológico de una ronda de juego

Una ronda estándar de Blackjack se ejecuta mediante una secuencia cronológica, compuesta por seis fases bien definidas:

Fase 1: Periodo de apuestas iniciales

Los jugadores colocan la cantidad de fichas deseada en su respectiva casilla de apuesta antes de que se inicie el reparto. Una vez que el crupier anuncia el cierre de apuestas, no se permite modificar, añadir ni retirar fichas del tapete.

Fase 2: Reparto inicial de cartas

El crupier distribuye las cartas desde el zapato siguiendo el sentido de las agujas del reloj:

- **Cartas del jugador:** Cada jugador recibe dos cartas consecutivas, ambas repartidas boca arriba.
- **Cartas del crupier:** El crupier se reparte una carta visible boca arriba y una segunda carta boca abajo, denominada 'carta oculta' o 'hole card' (en el estilo de juego americano).

Fase 3: Verificación antecedente de blackjack

Si la carta visible del crupier es un As o vale 10 puntos, la casa verifica internamente la carta cubierta antes de habilitar los turnos de la mesa:

- **Seguro:** Frente a un As visible, se ofrece la opción de realizar una apuesta de seguro como protección ante un Blackjack del crupier.
- **Resolución por Blackjack de la casa:** Si el crupier completa un 21 natural, la carta oculta se muestra al momento y la ronda concluye sin permitir más decisiones. Todos los jugadores que no empaten con otro Blackjack pierden su apuesta inicial.

Fase 4: Turno de decisiones del jugador

Si el crupier no posee un Blackjack inmediato, el turno de juego avanza individualmente con cada participante. Cada jugador analiza su mano frente a la carta visible del crupier y ejecuta sus opciones tácticas: Plantarse, Pedir carta, Doblar la apuesta o Dividir pares.

Fase 5: Turno obligatorio de la casa (Crupier)

Una vez que todos los jugadores en la mesa han completado sus turnos y mantenido sus manos válidas (sin haberse pasado de 21), el crupier procede a jugar su mano bajo un reglamento automático sumamente estricto:

- **Revelación de carta oculta:** El crupier revela su carta oculta para mostrar su puntuación inicial de dos cartas.
- **Regla de los 17 puntos:** El crupier no tiene libertad de decisión ni criterio personal. Debe pedir cartas obligatoriamente si su total es de 16 puntos o menos. Debe plantarse obligatoriamente tan pronto como alcance o supere los 17 puntos en total. Si al pedir cartas supera los 21 puntos, la casa se declara en 'Bust' y debe pagar automáticamente a todas las manos activas del tablero.

Fase 6: Comparación de puntuaciones y distribución de pagos

El crupier compara la puntuación de su mano final contra las manos de los jugadores activos. Se recogen las apuestas de las manos perdedoras, se devuelven las apuestas en caso de empate y se efectúan los pagos correspondientes según la tabla oficial.

Análisis de las acciones del jugador

Plantarse (Stand)

El jugador decide finalizar su turno sin solicitar cartas adicionales, fijando su puntuación actual para la comparación final. Esta acción es recomendable cuando se

cuenta con una mano dura alta (17 a 21 puntos) o cuando el crupier muestra una carta débil (4, 5 o 6) que le otorga una alta probabilidad estadística de pasarse.

Pedir carta (Hit)

El jugador solicita una nueva carta del mazo para incrementar su total. Se pueden pedir tantas cartas consecutivas como se desee, siempre que la suma acumulada no supere los 21 puntos. Si una carta adicional provoca que la suma sea mayor a 21, el jugador pierde automáticamente la apuesta en ese instante ('Bust').

Doblar la apuesta (Double Down)

Esta opción permite duplicar el importe de la apuesta inicial a cambio de aceptar una restricción rigurosa: el jugador recibirá única y exclusivamente una carta adicional, tras la cual su turno concluirá automáticamente. Es la jugada de mayor rendimiento financiero cuando el jugador inicia con un total de 9, 10 u 11 puntos frente a una carta descubierta débil del crupier (3, 4, 5 o 6).

Dividir la mano (Split)

Cuando las dos cartas iniciales del jugador son del mismo valor (por ejemplo, dos 8, dos Ases o un 10 y una Q), el jugador puede colocar una apuesta adicional idéntica a la inicial para separar ambas cartas en dos manos independientes. El crupier coloca una segunda carta sobre cada una y el jugador las juega secuencialmente como dos apuestas separadas.

Apuesta de seguro (Insurance)

Si la carta descubierta del crupier es un As, se ofrece la posibilidad de apostar hasta el 50% de la apuesta inicial como seguro. Si el crupier efectivamente revela un 10 o figura en su carta oculta completando un Blackjack, el seguro paga 2 a 1, neutralizando la pérdida de la apuesta principal.

Tabla de Pagos

Los rendimientos económicos en la mesa de Blackjack están estandarizados en los casinos internacionales:

Tipo de mano	Pago	Descripción
Victoria Estándar	1:1	Puntuación mayor a la del crupier (sin superar 21).v

Tabla de Pagos		
Los rendimientos económicos en la mesa de Blackjack están estandarizados en los casinos internacionales:		
Blackjack	3:2	Sumar 21 puntos con las primeras dos cartas (As + 10/Figura).
Empate(Push)	Devolución	Igualdad de puntos con el crupier.
Apuesta segura	2:1	Se activa si el crupier muestra un As y confirma tener Blackjack. Paga el **doble** de la apuesta de seguro.
Derrota	0/Pérdida	Exceder los 21 puntos o tener menor puntaje que el crupier.

Ejemplos de manos completas

Ejemplo 1.1: Victoria estándar por puntuación superior

Ocurre cuando el jugador obtiene una suma de puntos mayor a la del crupier sin sobrepasar el límite de 21 puntos.

Situación de la partida:

- **Apuesta inicial:** 100 fichas.
- **Cartas del jugador:** 10 de Diamantes y 9 de Tréboles.
- **Total del jugador:** 19 puntos.
- **Cartas del crupier:** 8 de Picas y 10 de Corazones.
- **Total del crupier:** 18 puntos.

Resultado: La mano del jugador (19 puntos) es superior a la del crupier (18 puntos). Al ganar con el pago estándar de 1:1, el jugador obtiene 100 fichas de ganancia y recupera su apuesta inicial, recibiendo un total de 200 fichas.

Ejemplo 1.2: Victoria por exceso del crupier (Bust del crupier)

Ocurre cuando el crupier está obligado a pedir cartas adicionales para alcanzar el mínimo de 17 puntos, pero termina superando los 21 puntos.

Situación de la partida:

- **Apuesta inicial:** 100 fichas.
- **Cartas del jugador:** As + 6 + 8 + 5.
- **Total del jugador:** 20 puntos.
- **Cartas del crupier:** 10 + 6 + Rey (10).
- **Total del crupier:** 26 puntos.

Resultado: El crupier se pasa de 21 puntos y pierde automáticamente. El jugador gana con un pago de 1:1, obteniendo 100 fichas de ganancia neta y un total devuelto de 200 fichas.

Ejemplo 2.1: Derrota inmediata por pasarse (Bust del jugador)

Ocurre cuando el jugador pide una carta adicional y su suma total supera los 21 puntos, perdiendo la mano al instante.

Situación de la partida:

- **Apuesta inicial:** 100 fichas.
- **Cartas del jugador:** 9 + 7 + Jota (10).
- **Total del jugador:** 26 puntos.
- **Cartas del crupier:** 7 de Diamantes (y carta oculta).
- **Total del crupier:** No requiere evaluación.

Resultado: Al sobrepasar los 21 puntos, el jugador cae en *Bust* de inmediato y la casa recolecta la apuesta de 100 fichas sin necesidad de que el crupier juegue su turno.

Ejemplo 2.2: Derrota por puntuación inferior frente al crupier

Ocurre cuando el jugador se planta con un puntaje válido, pero el crupier logra acumular una puntuación más alta sin exceder los 21 puntos.

Situación de la partida:

- **Apuesta inicial:** 100 fichas.
- **Cartas del jugador:** 10 y 7.
- **Total del jugador:** 17 puntos.
- **Cartas del crupier:** 10 y 9.
- **Total del crupier:** 19 puntos.

Resultado: El crupier alcanza 19 puntos y supera los 17 puntos del jugador. El jugador pierde la partida y el crupier retira los 100 fichas apostados

Ejemplo 3: Empate entre el jugador y el crupier

En una partida de Blackjack, ocurre un empate cuando el jugador y el crupier obtienen la misma cantidad de puntos, sin superar los 21. En esta situación, ninguno gana y el jugador recupera su apuesta inicial.

Situación de la partida:

- Apuesta inicial: 100 fichas.
- Cartas del jugador: 10 y 8.
- Total, del jugador: 18 puntos.
- Cartas del crupier: 9 y 9.
- Total, del crupier: 18 puntos.

Resultado:

Ambos participantes obtuvieron 18 puntos, por lo que la partida terminó en empate. El jugador recupera los 100 fichas que había apostado, sin obtener ganancias ni pérdidas.

Ejemplo 4: Victoria mediante Blackjack natural

Un Blackjack natural ocurre cuando el jugador consigue exactamente 21 puntos con sus primeras dos cartas, utilizando un As y una carta con valor de 10 puntos. Normalmente, esta combinación tiene un pago especial de 3:2, siempre que el crupier no tenga también Blackjack.

Situación de la partida:

- Apuesta inicial: 100 fichas.
- Cartas del jugador: As y Rey.
- Total, del jugador: 21 puntos.
- Cartas del crupier: 10 y 7.
- Total, del crupier: 17 puntos.

Resultado:

El jugador obtiene Blackjack natural y gana automáticamente frente a los 17 puntos del crupier.

Como el pago es de 3:2, por una apuesta de 100 fichas el jugador obtiene 150 fichas de ganancia. Al sumar la apuesta original, recibe un total de 250 fichas.

Ejemplo 5: Dividir o doblar la apuesta

Ejemplo 5.1: Dividir cartas (Split)

El jugador puede dividir sus cartas cuando ambas tienen el mismo valor, creando dos manos independientes y realizando una segunda apuesta.

Situación de la partida:

- Apuesta inicial: 100 fichas.
- Cartas iniciales: 8 de Corazones + 8 de Tréboles = 16 puntos.
- Acción: Dividir las cartas y apostar otras 100 fichas.
- Primera mano: 8 + 10 = 18 puntos.
- Segunda mano: 8 + 3 + 9 = 20 puntos.
- Crupier: 10 + 7 = 17 puntos.

Resultado:

Ambas manos superan los puntos del crupier. El jugador gana 200 fichas y recupera las 200 apostadas.

Pago final: 400 fichas.

Ejemplo 5.2: Doblar la apuesta (Double Down)

El jugador puede duplicar su apuesta inicial, pero solamente recibe una carta adicional antes de finalizar su turno.

Situación de la partida:

- Apuesta inicial: 100 fichas.
- Cartas iniciales: 5 + 6 = 11 puntos.
- Acción: Doblar la apuesta a 200 fichas.
- Carta adicional: 9 de Corazones.
- Total, del jugador: 20 puntos.
- Crupier: 4 + 10 + 5 = 19 puntos.

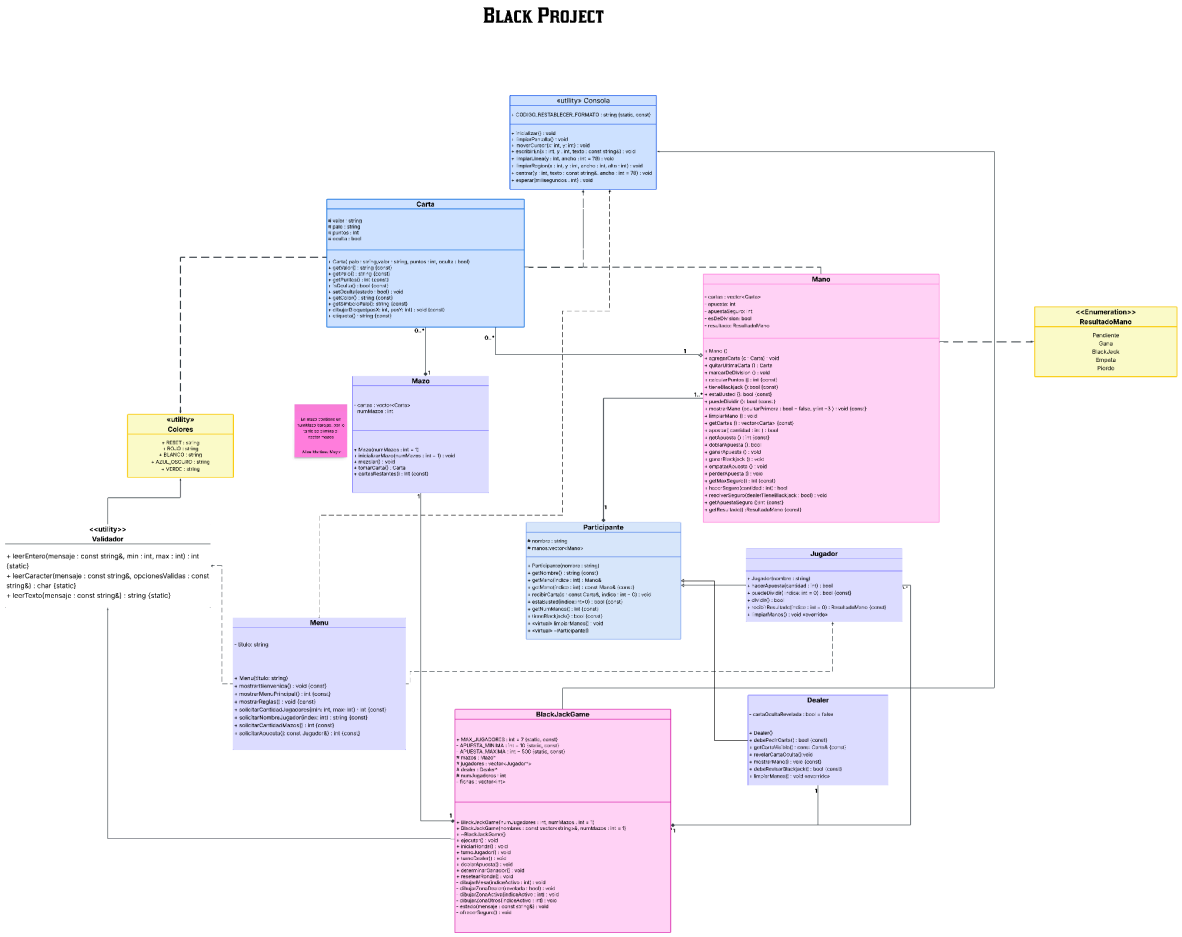
Resultado:

El jugador gana al superar los 19 puntos del crupier. Cómo dobló su apuesta, obtiene 200 fichas de ganancia y recupera las 200 apostadas.

Pago final: 400 fichas.

Diagramas de clases UML

Diagrama de clases



El diagrama utiliza la notación, + publico, # protegido,- privado y {const} al final de la firma de los métodos que no modifican el estado del objeto, de esta manera cada parte del diagrama de clases UML se implementa en C++. Las flechas con rombo relleno indican composición, la flecha con triángulo hueco indica herencia y la flecha punteada indica dependencia.

Relaciones de clases

Relación	Tipo	Cardinalidad	Descripción

Jugador→Participante	Herencia		Un jugador es un participante que guarda fichas, apuestas y seguro.
Dealer→Participante	Herencia		El dealer es un participante con la regla fija de pedir hasta 17.
Participante♦Mano	Composición	1 a 1..*	Un participante tiene una mano, o varias si se divide.
Mano♦Carta	Composición	1 a 0..*	Una mano contiene cero o más cartas
Mazo♦Carta	Composición	1 a 1..*	El zapato contiene cero o más cartas disponibles.
Blackjack Game♦Mazo	Composición	1 a 1	El juego posee un zapato de 6 mazos
Blackjack Game♦Jugador	Composición	1 a 1..*	El juego posee uno o más jugadores.
Blackjack Game♦Dealer	Composición	1 a 1	El juego posee un único dealer
Carta, Mano, Menú⇢Consola	Dependencia		Usan los colores y el control del cursor de la consola

Descripción de responsabilidades de las clases

Carta

Representa una carta individual de la baraja francesa que se usa en el juego de Blackjack. Guarda su palo, su valor (A, 2 a 10, J, Q, K), los puntos que vale en el juego y si está oculta (boca abajo) o visible. Permite consultar esos datos y cambiar su visibilidad. También decide su color (rojo para corazones y diamantes, blanco para tréboles y picas, azul oscuro si está oculta), devuelve su símbolo de palo y su etiqueta de texto, y se dibuja en la consola como un bloque en la posición indicada.

Mazo

La clase *mazo* representa el conjunto de cartas utilizadas en el juego de Blackjack. Se encarga de crear los mazos con sus 52 cartas, asignando sus valores y puntos correspondientes. Guarda las cartas en un *vector<Carta>* y permite mezclarlas aleatoriamente. También permite sacar cartas del mazo y consultar cuántas quedan disponibles durante la partida.

Mano

La clase Mano representa el conjunto de cartas que tiene un participante (jugador o dealer) durante una ronda de Blackjack. Guarda internamente un *vector<Carta>* con sus cartas. Permite agregar y quitar cartas, vaciar la mano al terminar la ronda y entregar sus cartas a otras clases con *getCartas*. También calcula el puntaje total, ajustando los ases entre 11 y 1 para no pasarse de 21, y dibuja las cartas en consola, con opción de ocultar la primera. El método *marcarDeDivision* queda preparado para la fase 2.

Participante

Es la clase base de *Jugador* y *Dealer*. Representa a cualquier persona que juega con cartas en la mesa. Guarda su nombre y un vector de manos, permite recibir una carta en una mano, consultar el nombre y las manos saber si el valor de la mano supera 21 o es un blackjack y vaciar sus manos al final de la ronda, que las clases hijas sobreescriben.

Jugador

Es la persona que apuesta contra la casa, hereda de *Participante* y guarda sus fichas disponibles, una apuesta por puesta por cada mano y la apuesta seguro, con *hacerApuesta* válida la apuesta y la descuenta de sus fichas, con *ganarApuesta(1:1), ganarBlackack (3:2)* lo redondea hacia abajo*,empatarApuesta* y *perderApuesta* actualiza el saldo según el resultado de la ronda. También gestiona las acciones especiales como: doblar, dividir y el seguro.

Dealer

Toma la representación de la casa, hereda de *Participante*, y solo agrega un atributo. que indica si su carta oculta ya fue relevante. Su responsabilidad es aplicar la regla del dealer con *debePedirCarta* y devuelve verdadero mientras su mano valga menos de 17 y falso o más. *BlackJackGame* usa este método para controlar el turno del dealer. Expone su carta visible, revela la carta oculta y muestra su mano.

BlackjackGame

Es la clase coordinadora de la partida. Guarda el zapato, los jugadores y el dealer, us metodos organizan el flujo del juego, *ejecutar*, *inicirRonda*, *turnoJugador*, *turnoDealer*, *determinarGanador* y *reseteaRonda*. Concentra la lógica del juego para que main no la contenga. En esta fase esta definida su estructura, pero se implementa hasta la fase 2.

Namespace Consola

Representa un conjunto de utilidades para controlar la consola, y por eso no es una clase con objetos sino un namespace de funciones inline y constantes. Guarda los códigos ANSI de color (RESET, ROJO, BLANCO, AZUL_OSCURO y VERDE) que usan las demás clases para pintar texto. Permite inicializar la consola en UTF-8 en Windows, limpiar la pantalla y mover el cursor a unas coordenadas (x, y). Carta y Mano dependen de él para dibujar las cartas en pantalla.

Plan de acción

A. Diagrama UML (Lucidchart)

#	Tarea	Responsable	Fecha	Depende de
A.1	Diseño inicial del UML por clase: atributos, métodos y relaciones	Cada integrante de la clase	Listo	Ninguno
A.2	Aplicar las sugerencias de la tutora y del profesor (vector<Mazo> y vector<Jugador> con multiplicidad 1..*, iniciarMazo(numMazos), doblarApuesta en blackJackGame)	Aline (Mazo), Julissa (blackJackGame) , Edson y Yoycel (revisan que las relaciones sigan coherentes)	14/10	A1
A.3	Revisar que las relaciones entre clases cuadren: agregación, composición,	Todos, en reunión por	17/10	A2

Plan de acción

	herencia y dependencia con consola y Validador	Discord compartiendo pantalla		
A.4	Actualizar el UML con los cambios que surjan al programar (firmas, parámetros nuevos)	Cada dueño de clase	29/30	Implementar las clases
A.5	Exportar el diagrama final como imagen legible e insertarlo en el documento	Julissa Mejias	29/30	A5
B. Repositorio y entorno				
#	**Tarea**	**Responsable**	**Fecha**	**Depende de**
1	Crear repositorio Blackjack-cpp, gitignore y agregar colaboradores	Edson	4/10	Ninguno
2	Aceptar invitación, clonar y crear estructura de carpetas	Todos	4/10	1
3	Definir convención de ramas (feature/nombre) y de mensajes de commit	Edson	4/10	1
C. Implementación por módulos				
#	**Tarea**	**Responsable**	**Fecha**	**Depende de**
4	Carta.h/cpp y consola.h (colores ANSI, limpiarPantalla, moverCursor)	Edson	5/10	2
5	Mazo.h/cpp (inicializar, mezclar, tomarCarta, cartasRestantes)	Yoycel Ortiz	22/10	4
6	Mano.h/cpp (agregarCarta, calcularPuntos con lógica del As, mostrarMano, limpiarMano)	Aline Martinez	20/10	4
7	Participante, Jugador y Dealer (herencia, apuestas, debePedirCarta())	Julissa Mejias	20/10	6
8	Pruebas unitarias de cada módulo con un	Cada dueño de	Mismo	4,5,6,7

Plan de acción

	main de prueba local	módulo	día que termina su módulo	
9	Merge de cada rama a main mediante Pull Request, revisado por otro integrante	Cada dueño, revisa otro	Día siguiente a su módulo	8
10	Validador (leerEntero, leerCaracter, leerTexto)	Edson Valle	21/10	Ninguna
11	Menu (bienvenida, reglas, pedir jugadores, mazos, nombre y apuesta)	Edson Valle	21/10	11
D. Documento Formal				
#	**Tarea**	**Responsable**	**Fecha**	**Depende de**
12	Descripción de responsabilidades de su clase (3 a 5 líneas: qué representa, qué guarda, qué hace)	Todos	24/10	UML
13	Portada, Introducción y Definición del problema	Julissa Mejias	25/10	Ninguna
14	Reglas del juego y Ejemplos 1 y 2 (victoria y derrota)	Edson Valle	26/10	Ninguna
15	Ejemplos 3, 4 y 5 (empate, blackjack, dividir o doblar)	Yoycel Ortiz	24/10	14
16	Minutas y Plan de acción	Aline Martinez	27/10	Ninguna
17	Avance de la implementación con capturas de pantalla	Yoycel Ortiz	25/10	Código
18	Insertar UML y descripciones, ensamblar y dar formato	Julissa Mejias	27/10	12 a 17
E. Exposición				

Plan de acción				
#	**Tarea**	**Responsable**	**Fecha**	**Depende de**
19	Crear la presentación	Aline Martinez	28/10	Todo lo anterior
20	apertura, reglas y ejemplos.	Julissa Mejias	Listo	Todo lo anterior
21	UML y responsabilidades.	Aline Martinez	Listo	Todo lo anterior
22	La demo	Yoycel y Edson	Listo	Todo lo anterior

Balance de carga		
Integrante	**Clases**	**Documento y apoyo**
Edson Valle	Consola, Carta, Validador y menú	Repositorio
Yoycel Ortiz	Mazo y BlackJack	Presentación y demo
Aline Martinez	Mano y main	Presentación y demo
Julissa Mejias	Participante, Jugador y Dealer	Guia de uso

Minutas de reuniones del Equipo.

Proyecto: Juego de BlackJack en C++
Modalidad: Las reuniones fueron llevadas en la plataforma de Discord, compartiendo pantalla en Visual Studio y Github.

Reunión del 4 de octubre de 2026

Presentes: Todos los integrantes del proyecto.
En esta primera reunión del equipo se definió la organización general y repartió el trabajo, se implementó un plan de desarrollo del proyecto. A Edson le correspondió la clase Carta junto con la implementación de la Consola.h. A Yoycel le correspondió la clase Mazo. A Aline le correspondió la clase Mano, con el cálculo de puntuación y la

gestión del As. A Julissa le correspondió la jerarquía de Participante, Jugador y Dealer, que usa herencia. La clase Juego quedó de implementarse entre todos, al igual que el main.cpp. Y finalmente se acordó utilizar Github con la guía del compañero Edson.

Reunión del 20 de octubre de 2026

Presentes: Todos los integrantes del proyecto.
Desarrollo: La reunión se llevó a cabo en la noche, con los integrantes compartiendo pantalla para explicar a los compañeros el avance de sus trabajos.

Acuerdos:

- Utilizar Lucidchart para implementar el diagrama UML de clases del proyecto. Le correspondió a cada integrante encargarse de sus respectivas clases para realizar el diagrama UML. Cada integrante iba comentando que podría mejorar en las clases de los demás compañeros, con el fin de garantizar que el código esté optimizado y si lo entiendan cada integrante.
- Trabajar en la rama Master, a pesar que se había creado una rama para cada integrante. Se presentaron unos problemas y para agilizar el trabajo se optó por trabajar en Master.

Reunión del 22 de octubre de 2026

Presentes: Todos los integrantes del proyecto.
Desarrollo: La reunión se llevó a cabo en la noche con la participación de todos los integrantes. Cada integrante iba compartiendo pantalla para explicar las modificaciones sugeridas por la tutora Maria del Mar. También hubo modificación en el código, entre ellas la incorporación de los archivos mazo.h y mazo.cpp realizada por el compañero Yoycel, también hubo correcciones de métodos. Por último, se realizó una distribución equitativa para el documento formal, y se compartió un enlace de documento para que cada uno realice su aporte correspondiente.

Acuerdos:

- La información deberá estar en el debido documento el 28 de octubre como fecha límite.
- Volver a consultar a la tutora de Programación 1 si tiene alguna sugerencia para mejorar nuestro código o UML.

Reunión del 27 de octubre de 2026

Presentes: todos los integrantes del proyecto.
Desarrollo: En la clase de 23 de octubre el profesor nos realizó sugerencias en UML, por lo tanto el codigo tambien debera ser modificado, entre las sugerencias realizadas por el profesor fueron: el juego deberá poder mezclar y que la clase carta no controla donde la deberia de poner (sus coordenadas posX y posY).
También nos comunicamos con el profesor por Teams ya que nos solicitó el UML para darnos más sugerencias. Por lo tanto la reunión realizada fue para discutir sobre el tema, además se trabajaron las modificaciones a las clases Carta, Mano, Participante y dealer.
También se empezó a implementar las bases de la clase blackJackGame.

Acuerdos:

- Empezar a unificar las clases en blackJackGame, para poder empezar a ejecutar el juego.
- Organizar una idea en concreto para el main.cpp según los requisitos que esta solicitando el profe.

Avance de la implementación:

Clase / módulo	Avance encontrado	Situación
Carta	Valores, palos, estado oculto y dibujo de cartas en consola.	Con código
Mazo	Zapato de 6 mazos (312 cartas), mezcla, reparto y conteo.	Probado en consola
Mano	Agregar y quitar cartas, sumar puntos, ajustar ases y mostrar manos.	Con código
Participante	Nombre, varias manos, detección de 21 y blackjack.	Con código
Jugador	Fichas, apuestas, pagos, doblar, dividir y seguro.	Con código
Dealer	Pedir con menos de 17, ocultar cartas y revisar blackjack.	Con código
Consola	Limpieza, colores y posicionamiento de elementos en consola.	Con código
BlackJackGame	Constructor y jugadores preparados; falta ejecutar la partida completa.	Pendiente integrar

Aspectos pendientes:

- Completar ejecutar(), iniciarRonda(), turnoJugador(), turnoDealer() y determinarGanador() en BlackJackGame.

- Conectar en BlackJackGame las clases ya programadas para que las rondas y los turnos funcionen juntos.

- Completar marcarDeDivision() en Mano y comprobar la división de cartas, apuestas y seguro durante una partida.

- Probar una partida completa. Por ahora se comprobó el zapato, el reparto inicial y la visualización en consola.

Evidencias: Programación y pruebas

Carta

Guarda el palo, valor, puntos y visibilidad de cada carta. También asigna símbolos, colores y dibuja las cartas en la consola.

Elementos observados: constructor, getPalo(), getValor(), getPuntos(), setOculta(), getColor(), dibujarBloque() y cartaEtiqueta().



```cpp
#include "Carta.h"
#include "Consola.h"
#include <iostream>

using namespace std;
// CONSTRUCTOR

// Inicializa las propiedades de la carta (palo, valor, puntos, posición inicial en la consola y estado oculta/visible)
Carta::Carta(string palo, string valor, int puntos, bool oculto) {
    this->palo = palo;
    this->valor = valor;
    this->puntos = puntos;

    this->oculto = oculto;
}


// MÉTODOS GETTER (Lectura de atributos)


string Carta::getPalo() const {
    return this->palo;
}

string Carta::getValor() const {
    return this->valor;
}

int Carta::getPuntos() const {
    return this->puntos;
}

bool Carta::isOculto() const {
    return this->oculto;
}


// MÉTODOS SETTER (Modificación de estado)



// Cambia la visibilidad de la carta (true = boca abajo, false = boca arriba)
void Carta::setOculta(bool estado) {
    this->oculto = estado;
}
string Carta::getSimboloPalo() const {
    if (palo == "Corazones" || palo =="corazones") return "♥";
    if (palo == "Diamantes" || palo == "diamantes") return "♦";
    if (palo == "Treboles" || palo == "treboles") return "♣";
    if (palo == "Picas" || palo == "picas") return "♠";
    return palo.substr(0, 1);
}

// DIBUJO Y FORMATO EN CONSOLA
//
// Determina el código de color ANSI según el estado o el palo de la carta
string Carta::getColor() const {
    if (oculto) return Consola::AZUL_OSCURO;
    string sim = getSimboloPalo();
    if (sim == "♥" || sim == "♦" || palo == "Corazones" || palo == "Diamantes") {
        return Consola::ROJO;
    }
    return Consola::BLANCO;
}

// Renderiza la carta gráficamente en la posición (posX, posY) de la consola mediante arte ASCII
void Carta::dibujarBloque(int posX, int posY) const {
    string color = getColor();
    // bordes de la carta
    string bordeSuperior = "┌────────┐";
    string bordeInferior = "└────────┘";
    // contenido de la carta
    Consola::moverCursor(posX, posY);
    cout << color << bordeSuperior << Consola::RESET;
    if (oculto) {
        //cuerpo de la carta oculta / fila de 1 a 5
        Consola::moverCursor(posX, posY + 1);
        cout << color << "│▒▒▒▒▒▒▒▒│" << Consola::RESET;
        Consola::moverCursor(posX, posY + 2);
        cout << color << "│▒▒▒▒▒▒▒▒│" << Consola::RESET;
        Consola::moverCursor(posX, posY + 3);
        cout << color << "│▒▒▒▒▒▒▒▒│" << Consola::RESET;
        Consola::moverCursor(posX, posY + 4);
        cout << color << "│▒▒▒▒▒▒▒▒│" << Consola::RESET;


    }
    else {
        string simPalo = getSimboloPalo();
        string valIzq = (valor.length() == 1) ? valor + " " : valor;
        string valDer = (valor.length() == 1) ? " " + valor : valor;
        //linea 1, valor de esquina superior izquierda
        Consola::moverCursor(posX, posY + 1);
        cout << color << "|" << valIzq << "      |" << Consola::RESET;

        //linea 2, simbolo del palo en el centro
        Consola::moverCursor(posX, posY + 2);
        cout << color << "|      " << simPalo << "      |" << Consola::RESET;

        //linea 3, valor de esquina inferior derecha
        Consola::moverCursor(posX, posY + 3);
        cout << color << "|      " << valDer << "|" << Consola::RESET;
    }
    // linea 4, borde inferior
    Consola::moverCursor(posX, posY + 4);
    cout << color << bordeInferior << Consola::RESET;
}
// Retorna una etiqueta textual legible
string Carta::etiqueta() const {
    if (oculto) {
        return "[Carta Oculta]";
    }
    return valor + " de " + palo;
}
```

Figura 1. Carta.cpp: datos de cada carta y representación en consola.

Estado: El dibujo y la visibilidad de las cartas se observan en las pruebas de ejecución.

Mazo

Forma un zapato con la cantidad de mazos indicada, mezcla las cartas y permite repartir una a una. También informa cuántas cartas quedan.

Elementos observados: inicializarMazo(), mezclar(), tomarCarta() y cartasRestantes().



```cpp
    shuffle(cartas.begin(),
        cartas.end(),
        generador);
}

// Sacar carta
Carta Mazo::tomarCarta() {

    Carta carta = cartas.back();

    cartas.pop_back();

    return carta;
}

// Cantidad de cartas restantes
int Mazo::cartasRestantes() const {

    return static_cast<int>(cartas.size());
}
```

Figura 2. Mazo.cpp: creación, mezcla y extracción de cartas.

Estado: Probado con 6 mazos (312 cartas); después de repartir 4 quedan 308.

Mano

Almacena las cartas de una mano, suma sus puntos y ajusta el valor de los ases si el total supera 21. También permite mostrar, quitar y limpiar cartas.

Elementos observados: agregarCarta(), quitarUltimaCarta(), calcularPuntos(), mostrarMano(), limpiarMano() y getCartas().



```cpp
#include "Mano.h"
#include "Consola.h"
#include <iostream>
#include <stdexcept>

using namespace std;

Mano::Mano() {}

void Mano::agregarCarta(Carta c) {
    cartas.push_back(c);
}


Carta Mano::quitarUltimaCarta() {
    Carta carta = cartas.back();// Obtiene la última carta de la mano
    cartas.pop_back();// Elimina la última carta de la mano
    return carta;
}


///Falta desarrollar este metodo para la fase 2, ya que le falta logica
void Mano::marcarDeDivision() {}


int Mano::calcularPuntos() const {
    int total = 0; // Suma de puntos de todas las cartas

    int numAses = 0;// Contador de ases para ajustar su valor si es necesario
    for (const Carta& carta : cartas) {
        total += carta.getPuntos();// Suma los puntos de la carta actual

        if (carta.getValor() == "A") {// Si la carta es un As, incrementa el contador de ases
            numAses++;// Incrementa el contador de ases
        }
    }
    // Ajuste de puntos para ases si el total excede 21
    while (total > 21 && numAses > 0) {
        total -= 10; // Cambia un As de 11 a 1
        numAses--;
    }
    return total;
}

    void Mano::mostrarMano(bool ocultarPrimera, int y) const {
        const int ANCHO_CARTA = 12;/// Ancho de cada carta en caracteres
            const int X_INICIAL = 5;/// Posición X inicial para dibujar la primera carta

        for (size_t i = 0; i < cartas.size(); ++i) {
            Carta carta = cartas[i];

            // Si se solicita ocultar la primera carta y estamos en la primera iteración, se marca como oculta
            if (ocultarPrimera && i == 0) {
                carta.setOculta(true);
            }
            int posX = X_INICIAL + static_cast<int>(i) * ANCHO_CARTA; // Calcula la posición X para la carta actual
            int posY = y;// Posición Y para dibujar la carta (constante para todas las cartas)

            //se utiliza metodo de carta.h
            carta.dibujarBloque(posX, posY);// Dibuja la carta en la consola

        }// Fin del bucle que recorre todas las cartas de la mano

        //se utiliza la funcion de consola.h
        Consola::moverCursor(1, y + 5); // Mueve el cursor a la siguiente línea después de dibujar la carta)
    }


    void Mano::limpiarMano() {
        cartas.clear(); // Elimina todas las cartas de la mano
    }

    vector<Carta> Mano::getCartas() const {
        return cartas; // Retorna el vector de cartas
    }
```

Figura 3. Mano.cpp: cálculo de puntos, dibujo y manejo de cartas.

Estado: La lógica básica funciona en la demostración; marcarDeDivision() todavía está sin desarrollar.

Participante

Es la clase base de Jugador y Dealer. Guarda el nombre y las manos y permite recibir cartas, consultar puntos y detectar si una mano se pasa de 21 o tiene blackjack.

Elementos observados: getNombre(), getNumMano(), getMano(), recibirCarta(), estaBusted(), tieneBlackjack() y limpiarManos().



```cpp
#include "Participante.h"

using namespace std;

Participante::Participante(const string& nombre) //guarda el nombre y crea la primera mano
{
    this->nombre = nombre;
    this->manos.push_back(Mano());
}

//getters

string Participante:: getNombre() const {
    return this->nombre;
}

//cuantas manos tiene el participante en este momento
int Participante::getNumManos() const {
    return static_cast<int>(this->manos.size());//
}

// CORRECCIÓN AQUÍ: Se agrega 'const' antes de Mano&
// Devuelve una referencia constante a la mano pedida.
// at() revisa el índice: si está fuera de rango lanza una excepción en lugar de leer memoria inválida.
const Mano& Participante::getMano(int indice) const {
    return this->manos.at(indice);

}

//sirve para saber si la mano se pasó de 21, si el índice no existe devuelve falso
bool Participante::estaBusted(int indice) const {
    if (indice < 0 || indice >= getNumManos()) {// Verificar si el índice es válido
        return false;
    }

    return this->manos.at(indice).calcularPuntos() > 21;// Verificar si la mano está "busted" (puntos > 21)
}

//agrega una carta a la mano indicada
void Participante::recibirCarta(const Carta& c, int indice) {
    if (indice < 0 || indice >= getNumManos()) { // si el índice es invalido no se hace nada,para evitar errores
        return;
    }

    this->manos.at(indice).agregarCarta(c);
}

//una mano es blackjack si tiene exactamente 2 cartas y suman 21
bool Participante::tieneBlackjack(int indice) const {
    if (indice < 0 || indice >= getNumManos()) {
        return false;
    }


    const Mano& m = this->manos.at(indice);

    return m.getCartas().size() == 2 &&
        m.calcularPuntos() == 21;
}

//prepara el participante para la egunda ronda, borrando todas las manos
void Participante::limpiarManos() {
    this->manos.clear();
    this->manos.push_back(Mano());
}
```

Figura 4. Participante.cpp: administración y validación de manos.

Estado: El código permite manejar varias manos y comprueba índices antes de acceder a ellas.

Jugador

Amplía Participante con fichas y apuestas. Incluye pagos por victoria o blackjack y acciones para doblar, dividir la mano y contratar seguro.

Elementos observados: hacerApuesta(), ganarApuesta(), ganarBlackjack(), perderApuesta(), empatarApuesta(), doblarApuesta(), dividir(), hacerSeguro() y resolverSeguro().



```cpp
#include "Jugador.h"
using namespace std;

//Contructor
Jugador::Jugador(string nombre, int fichas):Participante(nombre), fichas(fichas), apuestas(1, 0), apuestaSeguro(0) {
}

//Metodos de Apuesta
bool Jugador::hacerApuesta(int cantidad) {
    if (cantidad <= 0 || cantidad > fichas || apuestas[0] > 0) return false;
    fichas -= cantidad;
    apuestas[0] = cantidad;
    return true;
}
    //el jugador gana (1:1)
    void Jugador::ganarApuesta(int indice){
        if (indice < 0 || indice >= static_cast<int>(apuestas.size())) return;
        fichas += apuestas[indice] * 2;
    apuestas[indice] = 0;
    }

    //el jugador gana con blackjack (3:2). La division entera redondea hacia abajo.
    void Jugador::ganarBlackjack(int indice) {
        if (indice < 0 || indice >= static_cast<int>(apuestas.size())) return;
        int apuesta = apuestas[indice];
        fichas += apuesta + (apuesta * 3) / 2;
    apuestas[indice] = 0;
    }
    //el jugador pierde
    void Jugador::perderApuesta(int indice) {
        if (indice >= 0 && indice < static_cast<int>(apuestas.size())) apuestas[indice] = 0;
    }
    //el jugador empata
    void Jugador::empatarApuesta(int indice) {
        if (indice < 0 || indice >= static_cast<int>(apuestas.size())) return;
        fichas += apuestas[indice];
    apuestas[indice] = 0;
    }

    // Solo con las dos primeras cartas
    bool Jugador::puedeDoblar(int indice) const {
        return indice >= 0 && indice < getNumManos() && indice < static_cast<int>(apuestas.size())
        && getMano(indice).getCartas().size() == 2 && apuestas[indice] > 0
        && apuestas[indice] <= fichas;
    }


    // Dobla la apuesta actual (descuenta otra vez la misma cantidad)
    bool Jugador::doblarApuesta(int indice) {
        if (!puedeDoblar(indice)) return false;
        fichas -= apuestas[indice];
        apuestas[indice] *= 2;
        return true;
    }

    // Una sola division, solo con las 2 primeras cartas y del mismo VALOR (K y 10 sirven)
    bool Jugador::puedeDividir() const {
        if (getNumManos() != 1 || apuestas.empty() || apuestas[0] <= 0) return false;//solo se permite dividir si hay una sola mano y hay apuesta
        vector<Carta> cartas = getMano(0).getCartas();
        return cartas.size() == 2 && cartas[0].getPuntos() == cartas[1].getPuntos()
            && apuestas[0] <= fichas;//la segunda apuesta no puede ser mayor al saldo de fichas
    }

    // Crea la segunda mano y la segunda apuesta. Luego el juego reparte una carta a cada mano.
    bool Jugador::dividir() {
        if (!puedeDividir()) return false;//separa la mano y la apuesta
        fichas -= apuestas[0];//descuenta la segunda apuesta
        apuestas.push_back(apuestas[0]);//la segunda apuesta es igual a la primera
        Carta segunda = manos[0].quitarUltimaCarta();//quita la segunda carta de la primera mano y la pone en la segunda
        manos.push_back(Mano());//crea la segunda mano
```



```cpp
    bool Jugador::dividir() {
        if (!puedeDividir()) return false;//separa la mano y la apuesta
        fichas -= apuestas[0];//descuenta la segunda apuesta
        apuestas.push_back(apuestas[0]);//la segunda apuesta es igual a la primera
        Carta segunda = manos[0].quitarUltimaCarta();//quita la segunda carta de la primera mano y la pone en la segunda
        manos.push_back(Mano());//crea la segunda mano
        manos[0].marcarDeDivision();//marca la primera mano como dividida
        manos[1].marcarDeDivision();//marca la segunda mano como dividida
        manos[1].agregarCarta(segunda);//agrega la segunda carta a la segunda mano
        return true;
    }

    // Seguro,hasta la mitad de la apuesta original
    int Jugador::getMaxSeguro() const {
        return apuestas.empty() ? 0 : apuestas[0] / 2;
    }
    // Coloca la apuesta de seguro, si ya habia una no hace nada
    bool Jugador::hacerSeguro(int cantidad) {
        if (apuestaSeguro != 0 || cantidad <= 0 || cantidad > getMaxSeguro() || cantidad > fichas) return false;
        fichas -= cantidad;
        apuestaSeguro = cantidad;
        return true;
    }
    // Resuelve la apuesta de seguro. Si el dealer tiene blackjack, se paga 2:1, de lo contrario se pierde.
    void Jugador::resolverSeguro(bool dealerTieneBlackjack) {
        if (dealerTieneBlackjack) fichas += apuestaSeguro * 3;
        apuestaSeguro = 0;
    }

//Getters

    int Jugador::getFichas() const { return fichas; }//devuelve la cantidad de fichas disponibles

//devuelve la apuesta de la mano indicada, o 0 si no existe
int Jugador::getApuestaActual(int indice) const {
    return indice >= 0 && indice < static_cast<int>(apuestas.size()) ? apuestas[indice] : 0;
}

//prepara al jugador para la siguiente ronda, borrando todas las manos y apuestas
    void Jugador::limpiarManos() {
        Participante::limpiarManos();
        apuestas.assign(1, 0);
        apuestaSeguro = 0;
    }
```

Figura 5. Jugador.cpp: apuestas, pagos, división y seguro.

Estado: Estas funciones están escritas, pero falta probarlas juntas en una partida completa.

Dealer

Representa al crupier. Pide cartas si tiene menos de 17 puntos y puede ocultar o revelar su primera carta. También revisa cuándo corresponde comprobar blackjack o seguro.

Elementos observados: debePedirCarta(), getCartaVisible(), revelarCartaOculta(), mostrarMano(), ofrecerSeguro() y debeRevisarBlackjack().

Proyecto_Black (Ambito global)

```cpp
#include "Dealer.h"

using namespace std;

Dealer::Dealer() : Participante("Dealer") {

}

// El dealer pide carta mientras tenga menos de 17 puntos
bool Dealer::debePedirCarta() const {
    return getMano(0).calcularPuntos() < 17;
}

//la carta visible es la segunda que recibe dealer
Carta Dealer::getCartaVisible() const {
    vector<Carta> cartas = getMano(0).getCartas();
    return cartas[1];   // el juego debe repartir 2 cartas antes de llamarlo
}

// Revela la carta oculta del dealer, para mostrarla al final de la ronda
void Dealer::revelarCartaOculta() {
    cartaOcultaRevelada = true;
}

// Muestra la mano ocultando la primera carta hasta que se revele
void Dealer::mostrarMano() const {
    getMano(0).mostrarMano(!cartaOcultaRevelada);
}

// Devuelve verdadero si la carta visible es un As o un 10
bool Dealer::ofreceSeguro() const {
    if (getMano(0).getCartas().size() < 2) return false;
    return getCartaVisible().getValor() == "A";//solo se ofrece seguro si la carta visible es un As
}
// Devuelve verdadero si la carta visible es un As o un 10, para revisar blackjack
bool Dealer::debeRevisarBlackjack() const {//si no hay 2 cartas, no se puede revisar blackjack
    if (getMano(0).getCartas().size() < 2) {
        return false;
    }
    Carta visible = getCartaVisible();//la carta visible es la segunda que recibe dealer
    return visible.getValor() == "A" || visible.getPuntos() == 10;//si la carta visible es un As o un 10, se revisa blackjack
}

// Sobrescribe la mano de participante, además de vaciar las manos y reiniciar la carta oculta
void Dealer::limpiarManos() {
    Participante::limpiarManos();
    cartaOcultaRevelada = false;
}
```

Figura 6. Dealer.cpp: reglas para pedir, ocultar y revisar cartas.

Estado: La consulta de puntos usa getMano(0); la referencia incorrecta del documento anterior ya está corregida.

Consola

Apoya la presentación del juego en consola. Permite limpiar la pantalla, mover el cursor y utilizar colores para mostrar cartas e información.

Elementos observados: inicializar(), limpiarPantalla(), moverCursor() y constantes de color.



```cpp
#pragma once
#ifndef CONSOLA_H
#define CONSOLA_H

#include <iostream>
#include <string>

// Soporte para consolas Windows (UTF-8)
#ifdef _WIN32
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#endif

using namespace std;

namespace Consola {
    // Códigos ANSI para colores
    const string RESET = "\033[0m";
    const string ROJO = "\033[91m";
    const string BLANCO = "\033[97m";
    const string AZUL_OSCURO = "\033[34m";
    const string VERDE = "\033[92m";

    // Configura la consola al arrancar
    inline void inicializar() {
#ifdef _WIN32
        SetConsoleOutputCP(CP_UTF8);
#endif
    }

    // Limpia la pantalla y posiciona el cursor en el origen (1,1)
    inline void limpiarPantalla() {
#ifdef _WIN32
        SetConsoleOutputCP(CP_UTF8);
#endif
#ifdef _WIN32
        system("cls");
#else
        system("clear");
#endif
    }

    // Mueve el cursor a las coordenadas (x, y) de la consola
    inline void moverCursor(int x, int y) {
        cout << "\033[" << y << ";" << x << "H";
    }
}
```

Figura 7. Consola.h: funciones de presentación y posicionamiento.

Estado: Disponible y utilizado para dibujar las cartas en las pruebas.

BlackJackGame

Prepara los jugadores con sus fichas iniciales y un zapato de 6 mazos. Los métodos para iniciar rondas, manejar turnos y determinar el ganador todavía tienen comentarios en lugar de lógica.

Elementos observados: constructor, ejecutar(), iniciarRonda(), turnoJugador(), turnoDealer(), doblarApuesta(), determinarGanador() y resetearRonda().



```cpp
#include "blackJackGame.h"
#include <iostream>

BlackJackGame::BlackJackGame(int numJugadores) : numJugadores(numJugadores) {
    for (int i = 0; i < numJugadores; ++i) {
        jugadores.emplace_back("Jugador " + to_string(i + 1), 1000);
    }
    mazos.emplace_back(6); // zapato de 6 mazos (312 cartas) en un solo Mazo
}

void BlackJackGame::ejecutar() {
    // Lgica para ejecutar el flujo principal del juego
}

void BlackJackGame::iniciarRonda() {
    // Lgica para repartir cartas iniciales
}

void BlackJackGame::turnoJugador() {
    // Lgica para el turno del jugador (pedir, plantarse, etc.)
}

void BlackJackGame::turnoDealer() {
    // Lgica para el turno del dealer
}

void BlackJackGame::doblarApuesta() {
    // Lgica para doblar la apuesta del jugador
}

void BlackJackGame::determinarGanador() {
    // Comparacin de puntos entre el dealer y los jugadores
}

void BlackJackGame::resetearRonda() {
    // Limpiar las manos de los jugadores y el dealer para una nueva ronda
}
```

Figura 8. BlackJackGame.cpp: estructura de la partida pendiente de completar.

Estado: El constructor está preparado; falta implementar y conectar el flujo principal del juego.

Evidencias de ejecución: reparto inicial y presentación de cartas:

El archivo main.cpp realiza una demostración del zapato, el reparto inicial y el cálculo de puntos.

```cpp
#include "Consola.h"
#include "Mazo.h"
#include "Jugador.h"
#include "Dealer.h"
#include <iostream>

using namespace std;

// Demo: zapato de 6 mazos, barajar, repartir y calcular el valor de una mano
void correrDemo() {
    Consola::limpiarPantalla();

    Mazo zapato(6);
    cout << "1) Zapato creado: " << zapato.cartasRestantes() << " cartas \n";


    zapato.mezclar();
    cout << "2) Zapato barajado\n";

    Jugador jugador("Jugador 1", 1000);
    Dealer dealer;

    // Reparto inicial: 2 cartas para el jugador y 2 para el dealer
    jugador.recibirCarta(zapato.tomarCarta());

    // La primera carta del dealer se reparte oculta
    dealer.recibirCarta(zapato.tomarCarta());

    // Segunda carta del jugador
    jugador.recibirCarta(zapato.tomarCarta());

    // Segunda carta del dealer
    dealer.recibirCarta(zapato.tomarCarta());
    cout << "3) Reparto inicial hecho. Quedan " << zapato.cartasRestantes() << " cartas\n";


    cout << "\n" << jugador.getNombre() << ":\n";
    jugador.getMano(0).mostrarMano(false, 7);// Mostrar la mano del jugador sin ocultar cartas
    cout << "Puntos: " << jugador.getMano(0).calcularPuntos() << "\n";// Mostrar los puntos del jugador


    cout << "\nDealer (primera carta oculta):\n";
    dealer.getMano(0).mostrarMano(true, 15);// Mostrar la mano del dealer con la primera carta oculta
    cout << "Puntos reales: " << dealer.getMano(0).calcularPuntos() << "\n\n";// Mostrar los puntos reales del dealer (aunque la primera carta esté oculta)
}

int main() {
    Consola::inicializar();
    correrDemo();
    return 0;
}
```

Figura 9. main.cpp: prueba del reparto y de la puntuación.

Las siguientes pruebas muestran seis mazos mezclados, cuatro cartas repartidas y la primera carta del Dealer oculta.

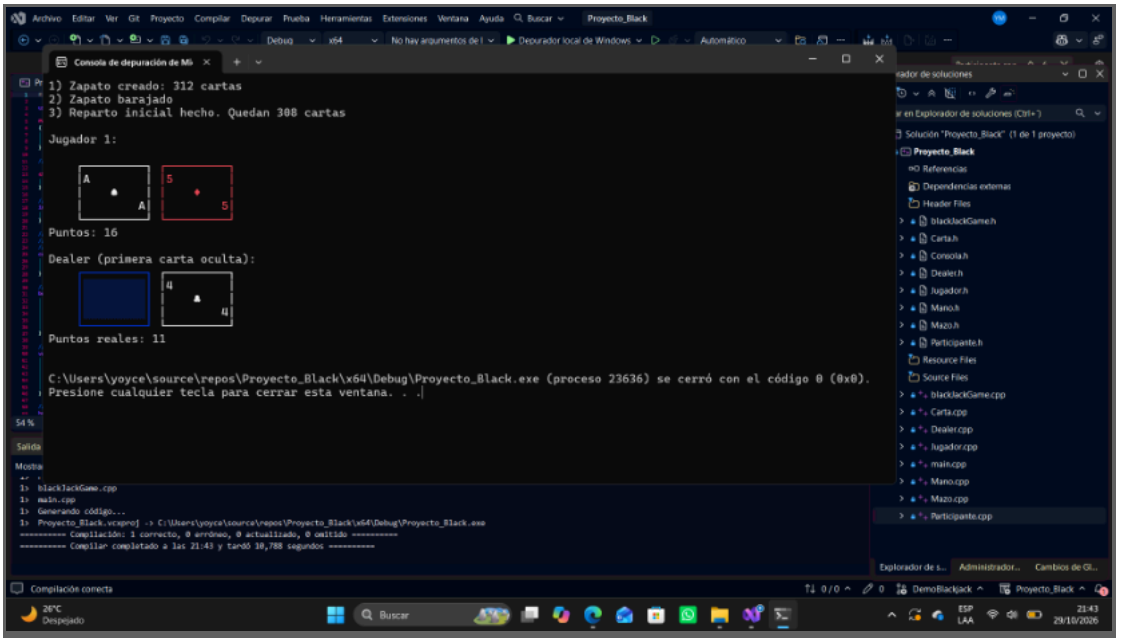


Figura 10. Prueba de ejecución: reparto inicial y cartas en consola.

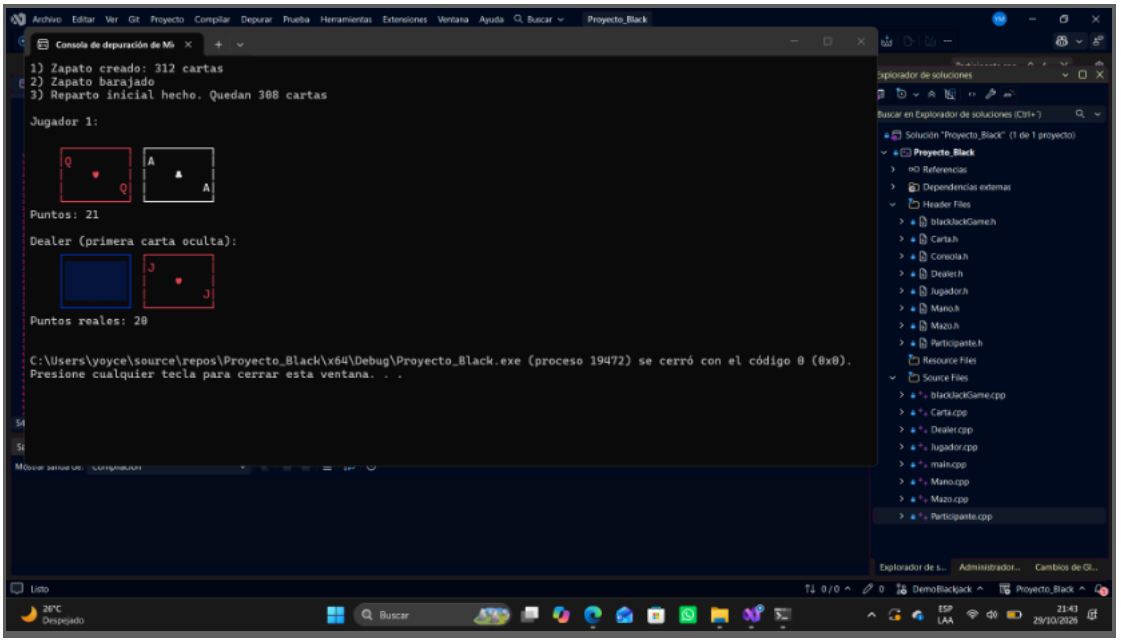


Figura 11. Prueba de ejecución: reparto inicial y cartas en consola.

Resultado: las dos pruebas terminaron sin errores de ejecución. Esto comprueba el reparto inicial, no una partida completa.